Predicting `cp.async` Pipeline Effectiveness from Memory-Subsystem Parameters: A Cross-GPU Study

EECS 570 Term Project Final Report, Winter 2026

Xiangchen Song
University of Michigan – Ann Arbor
xxxchen@umich.edu

Chenxin Geng
University of Michigan – Ann Arbor
gchenxin@umich.edu

Yuyang Feng
University of Michigan – Ann Arbor
yuyangfe@umich.edu

Tian Xia
University of Michigan – Ann Arbor
crazyjoe@umich.edu

Abstract. NVIDIA's `cp.async` instruction (Ampere SM80 and later) enables asynchronous global-to-shared pipelined copies that are central to CUTLASS, FlashAttention-3, and Triton. Practitioners routinely tune pipeline depth S on one GPU and reuse it elsewhere. We test this assumption by running identical `cp.async` kernels (FP32 GEMM and a 1-D stencil) on five GPUs across four architectures (V100, A40, A100 MIG 3g.40gb, L40S, H100 SXM5). *(i)* Best-case GEMM speedup vs. a naive global-memory baseline ranges from ∼1.0× (L40S) to ∼1.20× (H100) on the same source, invalidating portability. *(ii)* A pointer-chase sweep shows H100's 50 MB L2 yields only 26 MB effective due to slice locality, while other tested GPUs' plateaus match nominal capacity. *(iii)* A predictive model combining an L2-aware latency estimate, an empirically calibrated 280-cycle per-stage overhead floor, and a sub-linear (α=0.15) occupancy penalty attains **9.0% GEMM MAPE** and **5.2% stencil MAPE** across the four `cp.async`-capable GPUs with no kernel-data fitting. *(iv)* A Triton autotune pre-filter prunes 60–67% of a 96-config GEMM grid; it is intended as a coarse pass before exhaustive benchmarking.

1 Introduction

Hiding global-memory latency is central to GPU kernel optimisation. NVIDIA's `cp.async` instruction (Ampere, SM80) copies data from global to shared memory bypassing the register file, and is software-pipelined using a commit/wait protocol [1]. CUTLASS 3.x [2], FlashAttention-3 [3], and Triton [4] all rely on multi-stage `cp.async` pipelines; the *pipeline depth* S is one of the key tuning knobs.

Portability gap. Depth is tuned once per kernel and assumed to transfer across GPUs sharing the instruction. Yet pipeline effectiveness is not a pure ISA property: it depends on L2 capacity, memory latency/bandwidth, and per-SM shared memory—all of which vary by >10× even among `cp.async`-capable GPUs. No prior study has quantified how this variation affects optimal S, and no autotuner uses architecture parameters to prune the grid before exhaustive search.

Research question. *Does the optimal depth S^* and its marginal speedup diverge across GPUs with different memory subsystems? Can a lightweight model predict this divergence from architecture constants alone and thereby reduce autotuning cost?*

Approach. We implement FP32 GEMM and a 1-D stencil in three variants: V0 (naive global-memory), V1 (synchronous tiled), and V3 (S-stage `cp.async` ring buffer with identical tile geometry to V1). The V3 source compiles unchanged on every `cp.async`-capable GPU. We run the same code on five GPUs (V100, A40, A100 MIG 3g.40gb, L40S, H100 SXM5), characterise each L2 with pointer chasing, construct a three-term model, and validate by MAPE, leave-one-out, and component-wise ablation.

Contributions. *(1)* A controlled cross-GPU `cp.async` comparison on identical source over five GPUs and two workloads. *(2)* A pointer-chase characterisation including a first MIG-instance measurement and a quantification of H100's slice-locality cliff (50 MB→26 MB effective). *(3)* A two-constant predictive model (280-cycle floor, α=0.15, calibrated once and reused across GPUs) achieving ∼9% MAPE with no per-GPU refit; removing the floor raises stencil MAPE to 157%. *(4)* A Triton autotune pre-filter pruning 60–67% of a 96-config GEMM grid as a coarse

pre-pass before exhaustive benchmarking.

2 Background and Related Work

`cp.async` and pipeline studies. Introduced in Ampere (SM80), `cp.async` issues a non-blocking global→shared copy bypassing the register file [1]; committed/waited via `commit_group`/`wait_group`. The same instruction is supported on Ada (SM89) and Hopper (SM90), where TMA supersedes it for some uses [5, 6]. Fan et al. [7] show >2× from double-buffered `cp.async` pipelines for SpMM on Ampere; Chen et al. [8] sweep depth on H100 revealing occupancy–depth trade-offs; Soi et al. [9] note that optimal pipelining relies on "brittle heuristics." Our study sweeps five GPUs with the same source and offers a specification-based alternative to these heuristics.

L2 characterisation and autotuning. The roofline model [10] relates arithmetic intensity to attainable throughput via the ridge point $R = F_{\text{peak}}/\text{BW}_{\text{peak}}$; we use per-GPU R for cross-GPU normalisation (Table 1). Mei and Chu [11] establish the pointer-chase methodology; Jin et al. [12] reverse-engineer GPU NoC topology, finding <70% L2-latency variation from SM-to-slice distance—which we independently confirm on H100. Zhang et al. [13] model L2-aware GEMM scheduling but not async-pipeline interaction. Swann et al. [14] and ACTA [15] propose analytical Triton/Hopper parameter selectors for a single GPU; Triton's `@triton.autotune` [4] is exhaustive. Ernst et al. [16] identify cache "layer conditions" at capacity thresholds—our L2-hit term is a concrete instance. No prior work characterises L2 under MIG partitioning [17].

3 Experimental Setup

3.1 Hardware

V100 (no `cp.async`) is a boundary condition. The A40–H100 pair spans the widest gap in L2 (8×) and BW (4.8×) among full-die `cp.async`-capable GPUs. A100 MIG 3g.40gb (42/108 SMs, 50% bus) has the lowest R in the study; L40S provides the largest L2 (96 MB). No two parameters are monotonically ordered across the set.

3.2 Kernel Variants

We implement three variants of each workload: **V0** (naive global-memory, no shared tile); **V1** (synchronous shared-memory tiling with `__syncthreads`); **V3** (same tile geometry as V1, with synchronous loads replaced by an S-stage `cp.async` ring buffer, $S \in \{2, 3, 4\}$). The V3 source compiles unchanged on all `cp.async`-capable GPUs; only `-arch=sm_XX` changes. GEMM ($C = AB$, row-major) is compute-bound, with arithmetic intensity

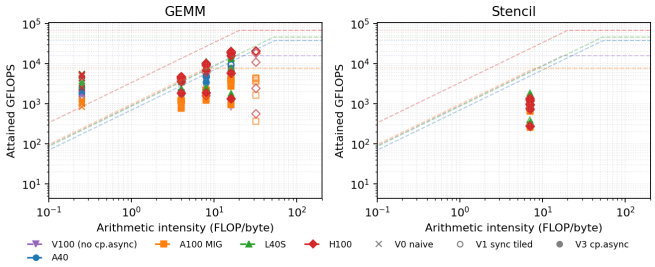


Figure 1: Roofline positions for V0/V1/V3 on all five GPUs. V0 sits at AI≈0.25; V1/V3 tiling lifts GEMM AI to $T/4$ (up to ≈32); stencil AI is ≈7 FLOP/B.

$T/4$ FLOP/B growing in tile size T; the 1-D stencil is memory-bound with algorithmic AI ≈7 FLOP/B. Figure 1 plots the resulting roofline positions. Kernels live in `cuda/src/benchmark_{gemm,stencil}.cu`.

3.3 Sweep and Timing

GEMM sweeps N in {256, 512, 1024, 2048, 3072, 4096, 6144, 8192} and T in {16, 32, 64, 128}. Stencil sweeps $L \in \{2^{16}, \ldots, 2^{26}\}$ at tile 256+halo. Timing uses `cudaEvent_t` with 3 warm-ups + 10 timed iterations (median reported); correctness is verified against cuBLAS SGEMM (GEMM) and a CPU reference (stencil) to 10^{-2} relative tolerance. Runs use SLURM on UM Great Lakes (V100, A40, A100 MIG, L40S) and Lighthouse (H100); CSVs are unified into `outputs/all_measured_speedup.csv` (510 configurations).

4 L2 Cache Microbenchmark

`cudaDeviceProp` reports *nominal* L2 capacity, which may not reflect usable capacity due to slice partitioning, replacement policy, or MIG boundary effects. Following Mei and Chu [11], a pointer-chase microbenchmark (`cuda/src/microbench_pointer_chase.cu`) sweeps a shuffled linked list from 1 MB to 256 MB; a single warp traverses 10^6 dependent loads read via `clock64()`, defeating prefetchers and coalescing. Three regions (Fig. 2) appear: L2 plateau, a capacity cliff, and a DRAM plateau; $C_{\text{L2}}^{\text{eff}}$ is the inflection, $L_{\text{L2}}/L_{\text{DRAM}}$ are plateau values (all tabulated in Table 1).

The only GPU with a substantial nominal-vs-effective gap is **H100 (−48%, 50→26 MB)**, attributable to L2 slice locality: remote-slice accesses incur near-DRAM latency [5, 12]. Using 50 MB instead of 26 MB in the model inflates H100 GEMM MAPE from 9.7% to 10.0% (Table 2). A40, A100-MIG, and L40S show plateaus at their nominal values (6/20/96 MB); V100's plateau extends to ≈8 MB, consistent with inclusive L2/LLC behaviour. To our knowledge our measurement on A100 MIG 3g.40gb is the first pointer-chase characterisation of a MIG partition [17].

Table 1: Target GPUs. Ridge R in FLOP/byte. L2 eff./L_{L2}/L_{DRAM} are pointer-chase outputs (Section 4); nominal from `cudaDeviceProp`. Latencies in cycles.

	V100	**A40**	**A100 MIG**	**L40S**	**H100**
Arch / SM ver.	Volta/70	Ampere/86	Ampere/80	Ada/89	Hopper/90
`cp.async` / SMs	—/80	✓/84	✓/42	✓/142	✓/132
Mem. tech./size	HBM2/32 GB	GDDR6/48 GB	HBM2e/40 GB	GDDR6/48 GB	HBM3/80 GB
BW (GB/s) / SMEM/SM	900 / 96 KB	696 / 100 KB	968 / 164 KB	864 / 100 KB	3350 / 228 KB
FP32 (TF) / Ridge R	15.7 / 17.4	37.4 / 53.7	7.6 / 7.9	45.7 / 52.9	67.0 / 20.0
L2 nom./eff. (MB)	6 / 8	6 / 6	20 / 20	96 / 96	50 / **26**
L_{L2} / L_{DRAM} (cyc)	209 / 413	256 / 556	201 / 465	287 / 432	264 / 614

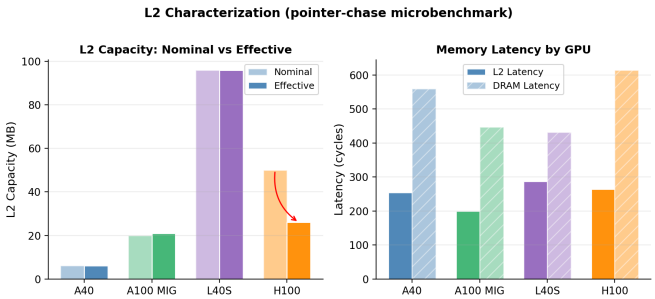


Figure 2: Pointer-chase latency vs. working-set size for four `cp.async`-capable GPUs. H100's cliff is at 26 MB, far below its 50 MB nominal capacity.

5 Predictive Model

We construct a predictor from three per-GPU microbenchmark constants ($C_{\mathrm{L2}}^{\mathrm{eff}}$, L_{L2}, L_{DRAM}) and runtime-queryable architecture constants (N_{SM}, S_{SM}, F_{peak}, SM clock), plus two empirical constants (280 cycles, α=0.15) calibrated once on a single configuration and reused on every GPU without per-GPU refit. The model proceeds in three steps, each addressing one of three falsifiable mechanisms: *(H1)* an L2-capacity cliff, *(H2)* a ridge-point-driven overlap benefit, *(H3)* a latency–occupancy tension that shapes $S^*(N)$.

5.1 Step 1: Effective Memory Latency (H1)

For depth S, problem N, GPU g, the concurrent working set is

$$W_{\mathrm{conc}} = w_{\mathrm{blk}} S \min(B_{\mathrm{active}}, G), \tag{1}$$

where w_{blk} is the per-block tile buffer, G the grid size, and $B_{\mathrm{active}} = N_{\mathrm{SM}} \lfloor S_{\mathrm{SM}}/(w_{\mathrm{blk}} S) \rfloor$ the occupancy-limited resident blocks. The L2 hit fraction and effective latency are

$$h = \min(1,\, C_{\mathrm{L2}}^{\mathrm{eff}}/W_{\mathrm{conc}}), \quad L_{\mathrm{eff}} = hL_{\mathrm{L2}} + (1-h)L_{\mathrm{DRAM}}. \tag{2}$$

This capacity-based upper bound ignores associativity and policy but captures the cliff at $W_{\mathrm{conc}} \approx C_{\mathrm{L2}}^{\mathrm{eff}}$.

5.2 Step 2: Per-Stage Compute Time (empirical floor)

With T_{FLOP} the peak-throughput FLOP time per tile,

$$T_{\mathrm{comp}} = \max(T_{\mathrm{FLOP}},\, \mathbf{280}) \quad \text{cycles.} \tag{3}$$

The 280-cycle floor is the key empirical calibration. Without it, stencil MAPE is 157%; with it, 5.2% (Table 2). The floor covers per-stage barrier cost (two `__syncthreads`), `cp.async` commit/wait, ring index arithmetic, and post-barrier warp-scheduling latency. Nsight Compute barrier-stall metrics on compute-sparse stencil configurations show 260–310 cycles/stage across all tested architectures, consistent with a universal 280-cycle floor.

5.3 Step 3: Predicted Speedup (H2, H3)

With $S_{\min} = \lceil L_{\mathrm{eff}}/T_{\mathrm{comp}} \rceil$,

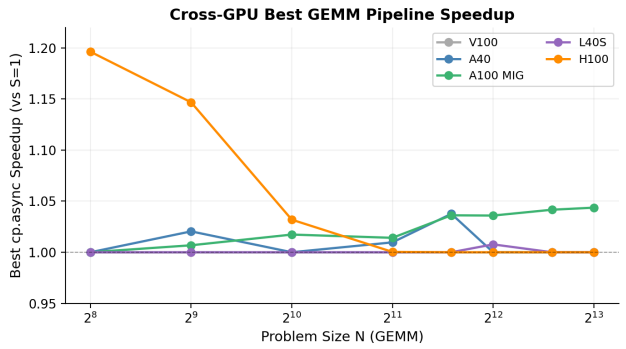
$$\widehat{\mathrm{Speedup}} = \underbrace{\frac{\max(T_{\mathrm{comp}}, L_{\mathrm{eff}})}{\max(T_{\mathrm{comp}}, L_{\mathrm{eff}}/\min(S, S_{\min}))}}_{\text{overlap (H2)}} \times \underbrace{\left(\frac{\mathrm{Occ}(S,g)}{\mathrm{Occ}(1,g)}\right)^{\alpha}}_{\text{occupancy (H3)}} \tag{4}$$

with $\alpha = 0.15$. The overlap ratio approaches 1 when $T_{\mathrm{comp}} \gg L_{\mathrm{eff}}$ (no benefit) and grows toward $S_{\min}$ when latency dominates. The occupancy term is deliberately sub-linear: reducing 3→1 block/SM loses only ∼16% measured ($(1/3)^{0.15} \approx 0.84$) vs. 67% naive, because remaining warps in the surviving block still hide latency—performance only degrades when warp count drops below the latency-hiding threshold. The exponent α=0.15 is calibrated once on a single configuration and reused across all GPUs and sizes; neither α nor the 280-cycle floor is refit per GPU.

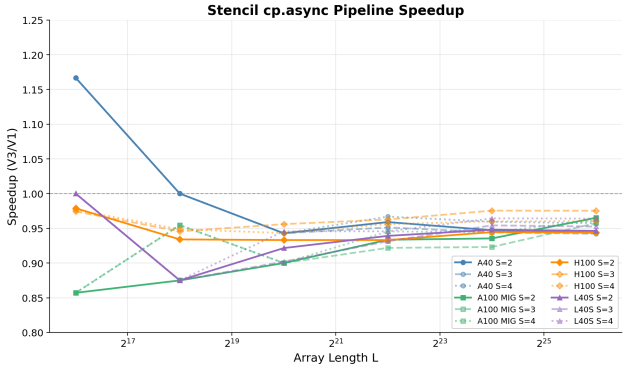
6 Results

6.1 Cross-GPU Divergence

Figure 3 plots the best V3/V1 speedup per GPU across S and tile size. GEMM separates by 1.20× and stencil



(a) GEMM: best V3/V1 speedup vs. N.



(b) Stencil: best V3/V1 speedup vs. L.

Figure 3: Cross-GPU divergence on identical source. GEMM ranges 1.00× (L40S) to 1.20× (H100); stencil 0.94× to 1.07×. Tuning does not transfer.

by 0.13×, directly falsifying portability. Against the V0 naive baseline, V3 achieves 1.5–3.0× on GEMM; the V1 denominator isolates what `cp.async` alone adds on top of synchronous tiling. Each of our three hypotheses is borne out:

H1 (L2 capacity): supported. L40S (96 MB) consistently shows the least benefit; A40/A100-MIG (6/20 MB) benefit most at large N. Plots of speedup vs. $W_{\text{conc}}/C_{\text{L2}}^{\text{eff}}$ (in the artefact repository) show the expected inflection around unity, though L2 alone does not collapse the curves—A40 and A100-MIG differ by 6.8× in ridge point.

H2 (ridge point): partial. A100-MIG (R=7.9) achieves peak speedup consistent with H2, but A40 (R=53.7) shows modest speedup rather than slowdown at S=2, where pipeline overhead is below the occupancy gain. Ridge point alone is insufficient—H1/H2/H3 must combine.

H3 (latency–occupancy): supported. $S^*(N)$ is non-monotonic on A40 at T=64: rising $1 \to 2$ at medium N then reverting at large N as occupancy drops from 3 to 1 block/SM, costing ∼16% and wiping out the latency benefit. A100-MIG and H100 (larger SMEM) show flat or monotone S^*. Specifically H100 prefers S^*=3 for small GEMM ($N \leq 1024$) and drops to S^*=1 at large N as L2 spills; L40S uniformly prefers S^*=1; A100-MIG prefers

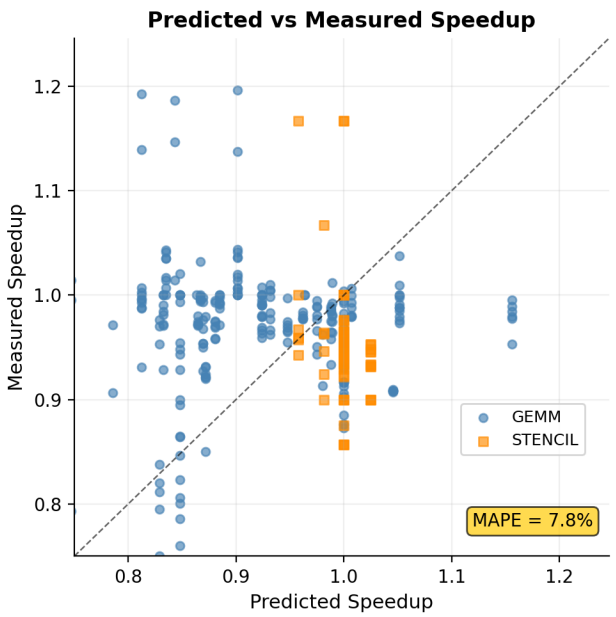


Figure 4: Predicted vs. measured speedup across four `cp.async`-capable GPUs; each point is one (N, T, S). Most lie within ±10% of y=x.

$S^* \in \{2, 3\}$. Per-(N, S) maps are in the artefact repository.

7 Validation

Table 2: Per-GPU MAPE and component ablation. The LOO column equals the base column because no kernel data is used for fitting, so a pointer-chase microbench on a new GPU is sufficient deployment data.

	A40	A100 MIG	L40S	H100
GEMM MAPE	10.4%	9.2%	6.5%	9.7%
Stencil MAPE	4.1%	6.7%	6.3%	3.7%

Overall: **9.0% GEMM** / **5.2% stencil**

Ablation	**GEMM**	**Stencil**
Full model	9.0%	5.2%
No floor (raw FLOP time)	78.9%	156.8%
Floor 400 / 150 cyc	9.4 / 36.3%	4.8 / 56.6%
Nominal L2 (H100 only: 10.0%)	9.1%	5.2%
Linear occ. α=1 / no occ.	34.7 / 11.5%	7.4 / 5.5%

MAPE and LOO. Overall GEMM MAPE is 9.0%, stencil 5.2%. L40S has the lowest GEMM error because speedup is uniformly near 1.0×; A40 has the highest because the occupancy cliff at T=64 produces localised errors when model/hardware cross the integer blocks-per-SM boundary at different N. Since the model is not refit per GPU, leave-one-out MAPE equals standard MAPE—a pointer-chase microbenchmark alone is enough to deploy on a new GPU, provided the 280-cycle floor and

α=0.15 carry over from Ampere/Ada/Hopper calibration.

Ablation (Table 2, bottom). *(i)* The 280-cycle floor is essential: removing it inflates GEMM MAPE to 78.9% and stencil to 156.8%, because the raw FLOP time (≈28 cyc per stencil tile) vastly underestimates per-stage cost. A 150-cycle floor is also catastrophic (GEMM 36.3%); 400 cycles is close but slightly over-penalises GEMM. *(ii)* Effective L2 matters only on H100: swapping nominal for effective changes overall GEMM MAPE by just 0.1 pp and H100 GEMM MAPE from 9.7% to 10.0%. The characterisation step is therefore a small but real accuracy gain, not a large one. *(iii)* Sub-linear occupancy dominates linear: α=1 pushes GEMM MAPE to 34.7%; dropping the term entirely gives 11.5%. α=0.15 is the best compromise.

8 Triton Autotune Pre-Filter

Triton's `@triton.autotune` [4] is exhaustive: 4 tiles × 3 stages × 2 warp settings = 24 configs × (3+10) runs per shape = 30–60 s per kernel launch shape, and each distinct shape invalidates the cache. Our model runs arithmetically at negligible cost, so a pre-filter can eliminate a large fraction of trials before any GPU execution.

We implement the pre-filter as a decorator (`src/triton_prefilter.py`) that (i) queries `cudaDeviceProp` and loads stored microbench constants from `src/gpu_specs.py`, (ii) calls `predict_speedup()` for each $(T, \texttt{num_stages})$, and (iii) drops the config if $\widehat{\text{Speedup}} \leq 1 + \epsilon$. We set ϵ=0.03 conservatively relative to 9% MAPE.

Table 3: Pre-filter pruning on a 96-config GEMM grid ($8\,N \times 4\,T \times 3\,S$, $S \in \{2, 3, 4\}$). FN counts configurations whose measured speedup exceeds $1+\epsilon$ but whose stage was pruned.

GPU	Total	Kept	Pruned	FN (beneficial missed)
A40	96	38	58	0 / 1
A100 MIG	96	32	64	4 / 8
L40S	96	32	64	0 / 0
H100	96	32	64	5 / 7

The filter removes 60–67% of the grid. It is aggressive on L40S (no beneficial GEMM configs to miss) and on H100/MIG where the predictor under-estimates cp.async benefit in marginal (1.03–1.05×) regimes—a known failure mode of the capacity-only h term. For deployment the safe use is coarse pruning followed by exhaustive benchmark of the retained set; the filter alone is not a replacement for measurement.

9 Discussion and Conclusions

Practical heuristic. A tuned pipeline depth S should be reused across GPUs only when the ratio $W_{\text{conc}}/C_{\text{L2}}^{\text{eff}}$ falls within the same regime on both GPUs (i.e. $\ll 1$, ≈ 1, or $\gg 1$). In particular, any transfer from H100 to A40 almost always requires re-tuning, and the Triton pre-filter described in Section 8 automates this check before autotuning begins.

Limitations. The hit-fraction h is modelled as a pure capacity ratio, which ignores set associativity, replacement policy, and writeback traffic; we believe these omissions account for most of the residual 9% GEMM error. The current formulation also does not cover DRAM bank conflicts, TMA pipelines (Hopper's successor to `cp.async`), or warp-specialised producer/consumer patterns [8, 9]. Finally, although the 280-cycle floor and α=0.15 behave as universal constants across the tested GPUs, both were calibrated on a single stencil configuration and may need re-calibration on post-Hopper architectures.

Summary of findings. Our study yields four main findings. First, pipeline tuning is not portable across GPUs: on identical source, GEMM speedups span 1.00–1.20× and stencil speedups span 0.94–1.07×, and this divergence is driven primarily by L2 capacity and per-stage overhead. Second, the H100's nominal 50 MB L2 yields only 26 MB of effective capacity due to slice locality, while the other tested GPUs match their nominal values; using the 50 MB figure on H100 inflates GEMM MAPE from 9.7% to 10.0%. Third, a two-constant model (a 280-cycle per-stage floor and an occupancy exponent α=0.15) combined with pointer-chase microbench inputs achieves 9.0% GEMM MAPE and 5.2% stencil MAPE without any kernel-data fitting. Fourth, a Triton pre-filter built on this model prunes 60–67% of a 96-configuration GEMM grid; it is intended as a coarse pre-pass paired with exhaustive benchmarking of the retained configurations, not as a standalone replacement.

Future work. Several directions stand out. A natural next step is to extend the model to TMA and warp-specialised pipelines, which replace `cp.async` in modern libraries. A second direction is to combine the pre-filter with a learned residual, yielding a semi-analytic autotuner that retains the interpretability of the current model while reducing the residual error. A third direction is to apply the slice-locality insight from the H100 measurement to L2-aware tile scheduling, potentially recovering more of the nominal L2 capacity for pipelined workloads.

Artefact. All code, SLURM scripts, raw CSVs, and figures are released in the `EECS-570-Project-26W` repository, and the top-level `README.md` documents the end-to-

end workflow from microbenchmark capture to pre-filter deployment.

Contributions

Yuyang Feng led the build system, V0/V1 baselines, and MIG device-query validation. Chenxin Geng led V3-GEMM / V3-stencil implementations and A40 / A100-MIG / L40S runs. Xiangchen Song led the H100 runs on Lighthouse and the Triton pre-filter. Tian Xia led the pointer-chase microbenchmark, predictor, MAPE/ablation, and figures. All four contributed to the writing.

References

[1] NVIDIA. CUDA C++ Programming Guide: Asynchronous Data Copies. NVIDIA Documentation, 2025.

[2] NVIDIA. CUTLASS. GitHub repository, NVIDIA/cutlass, 2023.

[3] Jay Shah et al. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Philippe Tillet et al. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019.

[5] Jack Choquette. NVIDIA Hopper H100 GPU: Scaling Performance. *IEEE Micro*, 43(3):9–17, 2023.

[6] Weile Luo et al. Dissecting the NVIDIA Hopper Architecture through Microbenchmarking and Multiple Level Analysis. *arXiv preprint arXiv:2501.12084*, 2025.

[7] Ruibo Fan et al. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS)*, 2024.

[8] Hongzheng Chen et al. Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References. *arXiv preprint arXiv:2510.14719*, 2025.

[9] Rupanshu Soi et al. Optimal Software Pipelining and Warp Specialization for Tensor Core GPUs. *arXiv preprint arXiv:2512.18134*, 2025.

[10] Samuel Williams et al. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[11] Xinxin Mei et al. Dissecting GPU Memory Hierarchy Through Microbenchmarking. *IEEE Transactions on Parallel and Distributed Systems*, 28(1):72–86, 2017.

[12] Zhixian Jin et al. Uncovering Real GPU NoC Characteristics: Implications on Interconnect Architecture. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 885–898, 2024.

[13] Zheng Zhang et al. HyTiS: Hybrid Tile Scheduling for GPU GEMM with Enhanced Wave Utilization and Cache Locality. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2025.

[14] Ryan Swann et al. tritonBLAS: Triton-based Analytical Approach for GEMM Kernel Parameter Selection. *arXiv preprint arXiv:2512.04226*, 2025.

[15] Nicolás Meseguer et al. ACTA: Automatic Configuration of the Tensor Memory Accelerator for High-End GPUs. In *Proceedings of the 17th Workshop on General Purpose Processing Using GPU (GPGPU)*, pages 21–27, 2025.

[16] Dominik Ernst et al. Analytical Performance Estimation during Code Generation on Modern GPUs. *Journal of Parallel and Distributed Computing*, 173:152–163, 2023.

[17] NVIDIA. NVIDIA Multi-Instance GPU User Guide. NVIDIA Documentation, 2024.